

Path Planning and Autonomous Parking Using Reinforcement Learning for a LiDAR-Equipped E-Puck Robot

Hugo Sousa^{*†}, Tiago Silva^{*†}, Zhixu Ni^{*†}

^{*}Faculty of Sciences, University of Porto (FCUP), Porto, Portugal

[†]Faculty of Engineering, University of Porto (FEUP), Porto, Portugal
up202305900, up202307202, up202205842

Abstract—Path planning and parking are challenging problems that are essential to autonomous robotics, with significant implications for the efficiency and adaptability of mobile robots. This paper presents a method that combines path planning with autonomous parking using reinforcement learning (RL) for an E-Puck robot equipped with a Light Detection and Ranging (LiDAR) sensor. The robot is trained in the Webots simulator, and three RL algorithms are evaluated under the same environment and reward function: Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3). The observation space includes LiDAR range readings together with the distance and the relative angle to the target. The robot navigates to and parks in one of two free bays within a row of eight, with the free bays and the robot's starting position randomized every episode to avoid overfitting. The reward function was designed to address specific failure modes observed during training, including collision, timeout, and circular motion. Experimental results show that the trained PPO and SAC policies guide the robot to its assigned bay while avoiding obstacles, reaching mean evaluation rewards above 98 after convergence, whereas TD3 converges more slowly and to a lower reward of approximately 87. These results indicate that reinforcement learning, combined with environment randomization and a carefully designed reward function, is a viable approach for combined path planning and parking in partially known environments, and that the choice of algorithm meaningfully affects the reliability of the learned behavior.

Index Terms—reinforcement learning, path planning, autonomous parking, proximal policy optimization, Webots, E-Puck, LiDAR, environment randomization

I. INTRODUCTION

A. Context

Path planning for robotic systems involves determining a route for a robot to navigate from a starting point to a target location while avoiding obstacles. When the target location is a designated parking space, this task extends naturally into autonomous parking, which additionally requires the robot to approach the target precisely enough to come to a stop within it rather than merely passing near it. Both tasks are critical in domains such as autonomous driving, warehouse automation, and service robotics [1]. Path planning and parking have traditionally relied on deterministic algorithms, such as Dijkstra's algorithm and A* for route planning, but the increasing complexity of real-world environments and the need

for adaptive behavior have made reinforcement learning (RL) an increasingly promising alternative.

B. Problem and Motivation

Deterministic path planning and parking algorithms generally assume that the environment is fully known and static. In practice, however, mobile robots are often required to operate in environments that are only partially known, that may change over time, or that contain configurations that were not anticipated at design time, such as which parking spaces happen to be free at a given moment. Automating path planning and parking through reinforcement learning can address this limitation, since RL allows an agent to learn a combined navigation and parking policy through direct interaction with its environment rather than through an explicit, hand-crafted map. This capability is particularly relevant in applications such as warehouse logistics and service robotics, where the set of obstacles and available parking or docking locations may vary between deployments. However, applying RL to this combined task introduces its own challenges, including the need for efficient exploration strategies, the design of a reward function that reliably guides learning toward both accurate navigation and a precise final stop, and the computational cost of training in high-dimensional observation spaces.

Among the state-of-the-art RL algorithms suited to this type of task are Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3). PPO balances exploration and exploitation by using a clipped objective function that ensures stable and efficient policy updates, which makes it well suited to the sequential decision-making required for path planning and parking. SAC and TD3 are off-policy continuous-control algorithms that have demonstrated strong sample efficiency on robotic control tasks. All three algorithms are evaluated and compared in this work.

C. Aims and Goals

This paper focuses on developing a combined path planning and parking policy for a simulated E-Puck robot using reinforcement learning, and on comparing how different RL algorithms perform on this task under matched conditions. The robot is trained within the Webots simulation platform, which

provides a versatile environment for testing and validating robotic behavior. The goal is to obtain a policy that is computationally efficient, robust to obstacles, able to generalize across different starting positions and target locations, and that reliably brings the robot to a stop within its assigned parking bay rather than only passing near it. To this end, the training environment randomizes both the robot’s starting position and the location of its assigned bay on every episode, and three RL algorithms, namely PPO, Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3), are trained under the resulting environment and compared in terms of convergence speed and final success rate.

D. Paper Structure

The remainder of this paper is organized as follows. Section II reviews related work on path planning and reinforcement learning in robotics. Section III details the proposed methodological approach, including the simulation setup and the design of the randomized training environment. Section IV presents the experimental evaluation, describing the reward function used and then comparing the performance of PPO, SAC, and TD3 under the resulting environment. Section V concludes the paper and discusses directions for future work.

II. RELATED WORK

The application of reinforcement learning to robotics has gained significant traction over the past few decades. RL allows robots to learn navigation and control policies through interaction with their environment, which makes it well suited to dynamic and complex tasks such as path planning. Foundational work on integrating learning algorithms with robotic control systems was carried out by Thrun and Schwartz [2] and by Kaelbling, Littman, and Moore [3], both of whom examined how function approximation and learning-based control could be combined with robotic decision-making.

Among the various RL algorithms applied to continuous control tasks, PPO has become particularly prominent. Introduced by Schulman et al. [4], PPO uses a clipped objective function to constrain policy updates, which improves training stability compared to earlier policy gradient methods. Its application to robotic path planning has produced encouraging results, allowing robots to navigate complex environments while maintaining stable training dynamics. Alongside PPO, off-policy actor-critic algorithms have also been widely applied to continuous control. Soft Actor-Critic (SAC), introduced by Haarnoja et al. [5], combines an actor-critic architecture with an entropy maximization objective that encourages exploration while remaining sample-efficient. Twin Delayed Deep Deterministic Policy Gradient (TD3), introduced by Fujimoto, van Hoof, and Meger [6], is an off-policy actor-critic algorithm that addresses overestimation bias in value-based continuous control through twin critic networks and delayed policy updates. These three algorithms, PPO, SAC, and TD3, are compared directly in the present work under the same robot navigation task.

Thrun, Burgard, and Fox [1] provide a broad overview of the principles underlying autonomous robot navigation, including the use of sensor models and probabilistic algorithms for mapping and path planning. This body of work establishes the classical, model-based perspective against which learning-based approaches such as the one proposed in this paper can be contrasted.

Beyond academic research, practical implementations have demonstrated the versatility of RL in real-world scenarios. The DoraPicker project, for example, presented an RL-based system that participated in the 2015 Amazon Picking Challenge, demonstrating that RL can be applied to complex manipulation tasks in unstructured environments [7].

Simulation platforms such as Webots [8] have further accelerated the development and validation of RL algorithms for robotics, since they allow researchers to train and test navigation policies across a wide range of scenarios before any deployment on physical hardware. The present work builds on this line of research by combining PPO, SAC, and TD3 with a LiDAR-equipped E-Puck robot in Webots to address path planning and parking jointly, and by randomizing both the robot’s starting position and its assigned target on every training episode, distinguishing it from prior work that applies a single RL algorithm to robot navigation under a fixed start-target configuration or that does not address the final parking step explicitly.

III. METHODOLOGICAL APPROACH

A. Simulation Setup

This work was conducted using the Webots robot simulator [8]. The robot used in all experiments is the E-Puck, a small differential-drive robot equipped with several sensors, including a LiDAR sensor, which was extensively used in this study. LiDAR is a remote sensing method that uses laser light to measure distance: the sensor emits laser pulses that reflect off nearby surfaces, and the distance to each surface is calculated from the time taken for the corresponding pulse to return. The sensor used in this work returns a point cloud representing the physical surfaces of the walls and objects surrounding the robot. The processing of this point cloud into a compact observation vector is described in Section IV.

B. Environment Design

The training environment consists of a single map containing eight parking bays arranged along a row, each large enough to hold one E-Puck robot. At every bay, a stationary robot of the same model is placed to represent a parked vehicle. At the start of each episode, two of the eight bays are selected uniformly at random and their occupying robots are removed from view, leaving exactly two bays free; the remaining six bays stay occupied and therefore act as obstacles. The robot under training is then assigned one of the two free bays as its target, and is required to navigate to that bay and stop within it while avoiding both the fixed walls of the environment and the six occupied bays.

In addition to randomizing which bays are free, the starting position of the robot under training is drawn at random from a fixed rectangular region of the map at the beginning of every episode, rather than being fixed to a single starting point. Combining randomized bay occupancy with a randomized starting position prevents the robot from learning a single memorized route between a fixed start and a fixed target, and instead requires it to generalize the underlying behavior of locating and approaching whichever bay is currently free. This randomization strategy replaces an earlier, staged curriculum design in which the robot trained on a fixed sequence of progressively harder static maps; the randomized single-environment design was found to be simpler to implement and to expose the robot to a substantially larger number of distinct start-target configurations within the same number of training episodes.

An episode terminates successfully once the distance between the robot and the center of its assigned bay falls below a fixed threshold, and terminates unsuccessfully if the robot's touch sensor registers a collision with a wall or with an occupied bay. Episodes are also truncated after a fixed number of simulation steps if neither condition is met, which prevents the robot from remaining idle or oscillating indefinitely during training. This proximity-based criterion treats parking as reaching a target position, without separately requiring a specific final orientation or that the robot remain stationary within the bay for a minimum duration; both of these additional requirements would more closely match how parking is typically understood, and are discussed as future work in Section V.

IV. EXPERIMENTAL EVALUATION

A. Reward Function Design

The reward function used in this work is presented in Algorithm 1. The action space consists of four actions: move forward, rotate left, rotate right, and move backward. The observation space consists of nine LiDAR readings sampled from the front 90-degree field of view of the sensor, together with the Euclidean distance to the target and the angle to the target, forming an eleven-element vector.

The design of the reward function was informed by behavioral problems observed during early training. In the initial stages of training, the robot would frequently collide with occupied bays or arena walls, or time out by wandering aimlessly without making progress toward the target. As training progressed, the robot discovered a local optimum of moving in circles, which avoided collisions and kept the episode alive but did not result in reaching the target. To break this behavior, two terms were added to the reward function: a bonus for visiting previously unvisited positions, which encourages spatial exploration, and a term proportional to the distance traveled at each step, which rewards forward motion. A per-step penalty for proximity to any wall or occupied bay was also introduced to discourage the robot from grazing obstacles even without triggering the touch sensor. A large penalty for remaining in the same neighborhood for more than

ten consecutive steps (-40) was included to directly penalize the circular-motion loophole. Together, these shaping terms guided the robot toward the intended behavior of navigating directly to the assigned bay and stopping within it.

Algorithm 1 Reward Calculation

```

1: advance simulation by one step
2:  $distance \leftarrow$  compute distance to target
3:  $prevDist \leftarrow$  distance to target at the previous step
4: if touch sensor is activated then
5:    $done \leftarrow true$ ;  $reward \leftarrow -100$ 
6:   return  $reward, done$ 
7: end if
8: if  $distance < distThreshold$  then
9:    $done \leftarrow true$ ;  $reward \leftarrow 100$ 
10: else
11:    $done \leftarrow false$ ;  $reward \leftarrow -0.5 \times distance$ 
12: end if
13:  $reward \leftarrow reward + 5 \times (prevDist - distance)$ 
14:  $reward \leftarrow reward - 0.8$ 
15:  $position \leftarrow$  current robot position from GPS, rounded to one decimal place
16: if  $position$  has not been visited before then
17:   mark  $position$  as visited;  $reward \leftarrow reward + 2$ 
18: end if
19:  $reward \leftarrow reward + 2 \times$  (distance traveled since previous step)
20:  $lidarFeatures \leftarrow$  processed LiDAR readings
21: for all  $feature$  in  $lidarFeatures$  do
22:   if  $feature \leq 0.02$  then
23:      $reward \leftarrow reward - 0.75$ 
24:   end if
25: end for
26: if  $distance \geq prevDist$  then
27:   increment  $sameSpotSteps$ 
28: else
29:    $sameSpotSteps \leftarrow 0$ 
30: end if
31: if  $sameSpotSteps > 10$  then
32:    $reward \leftarrow reward - 40$ ;  $sameSpotSteps \leftarrow 0$ 
33: end if
34:  $prevDist \leftarrow distance$ 
35: return  $reward, done$ 

```

B. Results

Using the randomized training environment described in Section III together with the reward function from Algorithm 1, the trained PPO policy was able to reach its assigned target while avoiding obstacles, as illustrated by the example trajectory in Fig. 1.

After training, the policy exhibited stable behavior across the evaluated scenarios. Fig. 2 shows a top-down view of the training environment, illustrating the layout of the eight parking bays and the lower arena region from which the robot spawns at the start of each episode.

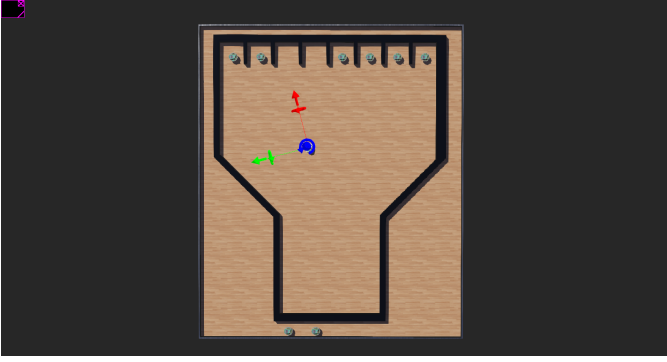


Fig. 1. The E-Puck robot (in blue) navigating toward the parking bays during inference. Two bays are free (visible gap in the top row); the robot must identify and reach one of them while avoiding the six occupied bays and the arena walls.

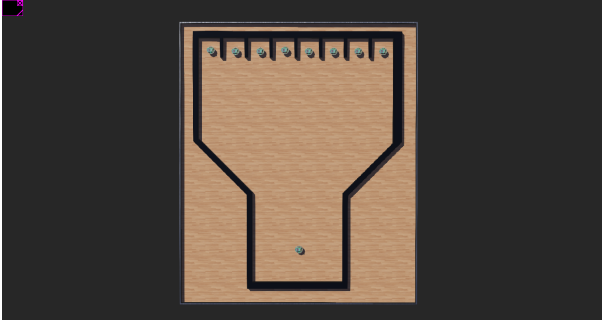


Fig. 2. Top-down view of the training environment in Webots. Eight parking bays are arranged along the top wall; each bay is occupied by a stationary E-Puck robot. At the start of every episode two bays are freed at random and the robot under training is spawned at a random position in the lower region of the arena.

Table I reports the evaluation metrics recorded at the best-performing checkpoints during training for each algorithm, as recorded by the evaluation callback. These snapshots correspond to the moments during training where each algorithm achieved its highest mean evaluation reward.

TABLE I
BEST EVALUATION CHECKPOINT PER ALGORITHM

Algorithm	Timestep	Mean reward	Std	Mean ep. length
PPO	160,000	105.31	1.24	88.0
SAC	80,000	98.20	3.14	106.4
TD3	140,000	87.50	3.54	108.6

Several observations follow from these results. SAC reached a mean evaluation reward above 93 as early as 60,000 timesteps, making it the fastest algorithm to converge to reliable performance. PPO required more timesteps to reach its best checkpoint but ultimately achieved the highest mean reward of the three algorithms (105.31 ± 1.24), along with the shortest mean episode length (88 steps), indicating that PPO’s policies completed parking maneuvers more efficiently on average once the policy had converged. TD3 achieved the lowest best-checkpoint reward (87.50 ± 3.54) despite being evaluated at 140,000 timesteps, consistent with the slower and

noisier convergence visible in the training curves in Fig. 3.

Despite these improvements, some limitations remain. No single scenario was identified in which the robot consistently struggled, but the residual difficulties that were observed, primarily occasional hesitation near obstacles, are expected to be further reduced by extending the duration of training on the randomized environment.

C. Comparison Between PPO, SAC, and TD3

Using the reward function described above, the same environment was used to train two further reinforcement learning algorithms, Soft Actor-Critic (SAC) and Twin Delayed Deep Deterministic Policy Gradient (TD3), so that their performance could be compared with that of PPO under identical environment dynamics and reward function. The PPO agent uses a discrete action space with three actions, namely moving forward, rotating left, and rotating right, whereas the SAC and TD3 agents use a continuous, one-dimensional action space that controls angular velocity, with linear velocity held constant. All three algorithms were trained with a learning rate of 3×10^{-4} , a discount factor of 0.99, and a budget of 900,000 environment timesteps; PPO used a smaller policy network with two hidden layers of 64 units, while SAC and TD3 used two hidden layers of 256 units each, following common practice for off-policy continuous-control algorithms.

Fig. 3 shows the smoothed episode reward and the corresponding success rate over the course of training for each algorithm, where an episode is considered successful if it terminates with a reward at or above 80, a threshold chosen to match the terminal reward awarded by the environment when the robot reaches its target. Table II summarizes the performance of each algorithm over the final 20% of its logged episodes, which is taken to approximate converged behavior.

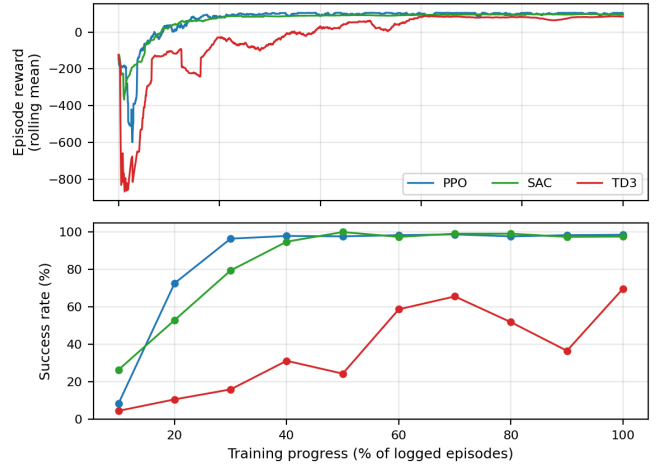


Fig. 3. Smoothed episode reward (top) and success rate (bottom) over the course of training for PPO, SAC, and TD3, expressed as a percentage of each algorithm’s logged training episodes. The success rate in the bottom panel is computed in ten equally sized bins of logged episodes.

Both PPO and SAC converge to a high success rate, above 97%, and do so within a comparable number of training

TABLE II
PERFORMANCE COMPARISON OVER THE ALL OF LOGGED EPISODES

Metric	PPO	SAC	TD3
Action space	Discrete	Continuous	Continuous
Episodes logged	4832	1178	1318
Success rate (%)	98.4	97.5	53.4
Mean reward	101.3	96.7	78.3
Median episode length (steps)	84	105	107

episodes, despite using different action space representations and network sizes. The PPO agent converges slightly faster in terms of logged episodes and reaches a marginally higher final success rate. The TD3 agent, by contrast, converges considerably more slowly and with substantially higher variance, and its success rate over the final portion of logged training remains close to 53%, well below that of the other two algorithms. This is consistent with the higher rate of collision-terminated episodes observed for TD3 relative to PPO and SAC. A complete account of these results would additionally require equalizing the number of environment steps recorded for each algorithm, since the number of logged episodes differs considerably between them, as shown in Table II; this is identified as a limitation of the present comparison and is discussed further in Section V.

The results in this section indicate that, for the present task, both the discrete action space used by PPO and the continuous action space used by SAC support reliable convergence, whereas TD3 with the hyperparameters used in this work is comparatively less reliable within the same training budget¹.

V. CONCLUSIONS AND FUTURE WORK

In this work, the application of reinforcement learning to combined path planning and parking in robotic systems was explored, using PPO, SAC, and TD3 within the Webots simulation platform to control an E-Puck robot equipped with a LiDAR sensor. The experimental results demonstrate that the proposed approach, combining a randomized training environment with the reward function described in Section IV, allows the robot to navigate toward a randomly assigned parking bay while avoiding obstacles. The comparison between algorithms further shows that PPO and SAC reach a comparably high success rate, whereas TD3, under the same environment and training budget, converges more slowly and to a lower success rate. These results indicate that reinforcement learning is a viable approach for enhancing the autonomy of mobile robots in partially known environments, and that the choice of algorithm can meaningfully affect the reliability of the learned behavior.

Despite these promising results, some challenges remain. The robot occasionally became stuck near obstacles for short periods, suggesting that the reward function and exploration strategy could be further refined. The TD3 agent did not reach the same level of performance as PPO and SAC within the available budget, and it remains unclear whether this is due

to the algorithm itself, its hyperparameters, or the unequal number of environment steps recorded across algorithms in this comparison.

To build on these findings, four directions for future research are proposed. First, the success criterion itself could be made more representative of parking in the fuller sense of the term, by additionally requiring the robot to reach a specific final orientation within its assigned bay and to remain stationary there for a short minimum duration, rather than terminating the episode as soon as the target position is reached. Second, the reward function could be refined further to discourage the robot from becoming stuck, for instance through penalties for repeated actions at the same location or through intermediate sub-goals. Third, the comparison between PPO, SAC, and TD3 should be repeated with an equal number of environment timesteps recorded for every algorithm, to determine whether additional hyperparameter tuning narrows the gap observed for TD3. Fourth, the learned policies could be transferred to a physical E-Puck robot and extended to scenarios involving multiple robots, moving obstacles, or additional sensing modalities.

By pursuing these directions, future work can move toward more robust and adaptable robotic systems capable of solving increasingly complex path planning and parking tasks across a wide range of applications.

REFERENCES

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [2] S. B. Thrun and A. Schwartz, "Issues in using function approximation for reinforcement learning," in *Proc. 1993 Connectionist Models Summer School*, vol. 6, 1993, pp. 255–263.
- [3] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, pp. 237–285, 1996.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*. JMLR.org, 2017, pp. 222–235.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*. PMLR, 2018, pp. 1861–1870.
- [6] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*. PMLR, 2018, pp. 1582–1591.
- [7] T. Zhang, X. Sun, Y. Xu, S. Zhang, C. Liu, H. Yang, J. Pan, and H. He, "DoraPicker: An autonomous picking system for general objects," *IEEE Trans. Autom. Sci. Eng.*, vol. 13, no. 4, pp. 1440–1452, 2016.
- [8] O. Michel, "Cyberbotics Ltd. Webots™: Professional mobile robot simulation," *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 40–43, 2004.

¹Source code repository: <https://github.com/zni0218/E-Puck-Robot>.